DEVSU

Arquitectura Empresarial y Diseño de Solución para Plataforma de Banca por Internet

Propuesta de Arquitectura de Software

Presentado por:

Andrés David Montaña Ardila

Ciudad:

Bogotá, Colombia

Fecha:

Abril de 2026

TABLA DE CONTENIDO

1. Introducción 3
2. Resumen ejecutivo de la solución 4
3. Interpretación del problema y alcance 5
4. Requerimientos funcionales 6
5. Requerimientos no funcionales 7
6. Supuestos arquitectónicos 8
7. Principales decisiones de arquitectura 9
8. Justificación de tecnologías, patrones y enfoque 13
9. Diseño de arquitectura propuesto 15
10. Diagrama C4 de Contexto 17
11. Diagrama C4 de Contenedores 18
12. Diagrama C4 de Componentes 19
13. Arquitectura de autenticación y autorización 20
14. Arquitectura de onboarding con reconocimiento facial 23
15. Arquitectura de integración con sistemas Core y sistemas complementarios 26
16. Arquitectura de notificaciones 28
17. Arquitectura de auditoría y trazabilidad 29
18. Estrategia de datos y acceso a datos 31
19. Estrategia de clientes frecuentes / optimización de desempeño / caching 33
20. Alta disponibilidad, tolerancia a fallos, DR y auto-healing 34
21. Seguridad, cumplimiento normativo y controles 36
22. Observabilidad, monitoreo, alertamiento y operación 38
23. Estrategia de nube (AWS) 41
24. Manejo de costos y trade-offs 43
25. Riesgos de arquitectura y mitigaciones 44
26. Conclusión final 45
27. Referencias 46

Introducción

La digitalización de los servicios financieros ha elevado de forma significativa las expectativas de los clientes respecto a disponibilidad, seguridad, inmediatez y experiencia de usuario en los canales digitales. En este contexto, las plataformas de banca por internet ya no pueden concebirse únicamente como interfaces de consulta o transacción, sino como ecosistemas tecnológicos altamente integrados, resilientes y gobernados, capaces de operar sobre sistemas Core existentes sin comprometer la trazabilidad, la continuidad operativa ni el cumplimiento normativo.

El presente documento desarrolla una propuesta de arquitectura empresarial y diseño de solución para una plataforma de banca por internet orientada a una entidad financiera que requiere habilitar dos canales digitales principales: una aplicación web tipo SPA y una aplicación móvil multiplataforma. La solución planteada contempla la consulta de productos y movimientos, la ejecución de transferencias entre cuentas propias e interbancarias, el soporte de procesos de onboarding digital con reconocimiento facial, la emisión de notificaciones por múltiples canales y el registro integral de auditoría sobre las acciones realizadas por el cliente.

Desde el punto de vista arquitectónico, la propuesta parte del principio de que el Core bancario debe mantenerse como sistema de registro, mientras que la capa digital debe concentrarse en la orquestación de procesos, la optimización de la experiencia de usuario, la integración desacoplada con sistemas internos y externos, la protección de identidades, la observabilidad y la capacidad de evolución del ecosistema. Bajo esta perspectiva, se adopta el modelo C4 como marco de representación de la solución, permitiendo describir la arquitectura desde tres niveles complementarios: contexto, contenedores y componentes.

Adicionalmente, el documento incorpora decisiones de diseño alineadas con prácticas modernas de arquitectura de software para entornos bancarios, incluyendo autenticación y autorización basadas en OAuth 2.0, separación clara de responsabilidades, integración mediante capas anticorrupción, publicación de eventos para auditoría y notificaciones, uso selectivo de mecanismos de caching para clientes frecuentes, y despliegue cloud-ready sobre AWS con criterios de alta disponibilidad, tolerancia a fallos, recuperación ante desastres y monitoreo continuo.

1. Resumen ejecutivo de la solución

Se propone una arquitectura **orientada a dominios, desacoplada y cloud-ready** para una plataforma de banca por internet que atiende dos canales digitales: una **SPA web** y una **aplicación móvil multiplataforma**. La solución se apoya en un **Authorization Server corporativo** compatible con OAuth 2.0 y federación, integra un **servicio externo de onboarding biométrico**, consume capacidades del **Core bancario** y de un **sistema complementario de cliente**, y publica eventos hacia una **plataforma de auditoría/SIEM** y proveedores de notificación.

La arquitectura se estructura en cinco capas. La primera es la **capa de acceso y canal**, con CDN/WAF/API Gateway y dos BFFs, uno por canal. La segunda es la **capa de negocio**, compuesta por servicios de consulta de cliente/productos, movimientos, transferencias, notificaciones y auditoría. La tercera es la **capa de integración/anticorrupción**, que encapsula el Core, el sistema complementario, la red interbancaria y los proveedores externos. La cuarta es la **capa de datos**, con una base de datos operativa para metadatos de canal y procesos, una base append-only de auditoría, almacenamiento de evidencias y un read model/cache para clientes frecuentes. La quinta es la **capa operativa**, con observabilidad, seguridad, resiliencia y automatización.

La decisión central de diseño es que el **Core bancario permanezca como sistema de registro**, mientras la capa digital agrega valor mediante orquestación, experiencia de usuario, seguridad contextual, auditoría, notificaciones y optimización de lectura. Esto evita construir una “sombra” del Core que genere inconsistencia, y al mismo tiempo habilita escalabilidad independiente de los canales.

Para autenticación, la recomendación principal es usar **Authorization Code + PKCE**; en móvil, con navegador del sistema o browser tab, y en web, endurecido mediante patrón **BFF** para reducir exposición de tokens en el navegador. Esta recomendación es consistente con las prácticas vigentes de OAuth para clientes públicos y apps nativas, y con la guía actual de seguridad OAuth y federación digital.

En nube, la propuesta se alinea con AWS y sus principios Well-Architected, desplegando los componentes críticos en múltiples Availability Zones y previendo una estrategia de DR regional. AWS documenta explícitamente que las AZ son ubicaciones

aisladas dentro de una región y que desplegar en múltiples AZ mejora disponibilidad; Aurora, además, ofrece replicación multi-AZ y opción de Global Database para resiliencia cross-region.

2. Interpretación del problema y alcance

2.1 Qué pide exactamente el ejercicio

El problema plantea diseñar una solución de banca por internet para que los usuarios puedan:

- consultar historial de movimientos;
- realizar transferencias entre cuentas propias e interbancarias;
- autenticarse mediante un servicio corporativo basado en OAuth 2.0;
- completar onboarding desde la app móvil con reconocimiento facial;
- recibir notificaciones por al menos dos canales;
- operar sobre información proveniente de un Core y de un sistema complementario;
- registrar auditoría completa de acciones del cliente;
- optimizar desempeño para clientes frecuentes;
- garantizar seguridad, alta disponibilidad, monitoreo, recuperación ante desastres y operación cloud.

2.2 Qué entregaré en esta propuesta

Esta respuesta se redacta como **propuesta de arquitectura**, no como implementación ya construida. Incluye:

- interpretación formal del problema;
- requerimientos funcionales y no funcionales;
- supuestos arquitectónicos;
- decisiones con justificación y alternativas;
- diseño propuesto;
- desarrollo de los tres niveles C4 pedidos;
- arquitectura de autenticación, onboarding, integración, notificaciones, auditoría, datos, caching, HA/DR, seguridad, observabilidad y nube;

- estrategia de costos y riesgos;
- código Python completo para generar los diagramas en PNG.

2.3 Alcance funcional y técnico

Dentro del alcance quedan los canales digitales, la integración con los sistemas requeridos, la arquitectura de seguridad, los patrones de integración, la capa de auditoría, el modelo operativo y el diseño cloud.

Fuera de alcance quedan, salvo mención como integración externa, el reemplazo del Core bancario, la implementación detallada de motores AML/fraude, los procesos manuales de backoffice, la parametrización regulatoria país por país y el diseño físico detallado de redes corporativas on-premise.

3. Requerimientos funcionales

1. El sistema debe exponer una **SPA web** para clientes digitales.
2. El sistema debe exponer una **aplicación móvil multiplataforma**.
3. Los usuarios deben poder **autenticarse** mediante un servicio corporativo basado en OAuth 2.0.
4. La app móvil debe permitir **onboarding de nuevos clientes** con reconocimiento facial y validación documental.
5. El sistema debe permitir **consulta de datos básicos del cliente y de sus productos**.
6. El sistema debe permitir **consulta de movimientos**.
7. El sistema debe permitir **transferencias entre cuentas propias**.
8. El sistema debe permitir **transferencias interbancarias**.
9. La información del cliente debe obtenerse de dos fuentes:
   - un **Core** con datos básicos, productos y movimientos;
   - un **sistema complementario** para datos ampliados.
10. El sistema debe **notificar movimientos y operaciones**, usando al menos dos canales.
11. El sistema debe **registrar auditoría completa** de las acciones del cliente.

12. Debe existir una solución de **persistencia auxiliar** para optimizar la experiencia de clientes frecuentes.

4. Requerimientos no funcionales

4.1 Seguridad

La solución debe diseñarse bajo principio de **security by design**, con autenticación robusta, MFA/step-up para operaciones de riesgo, cifrado en tránsito y en reposo, segregación de funciones, protección de APIs, trazabilidad, gestión segura de secretos y controles contra abuso automatizado.

4.2 Disponibilidad y resiliencia

Se requiere una plataforma con **alta disponibilidad**, tolerancia a fallos de infraestructura y fallos parciales de dependencias, mecanismos de reintento controlado, timeouts, circuit breakers, colas de desacoplamiento y recuperación ante desastres.

4.3 Escalabilidad

Los componentes deben escalar de forma independiente. El patrón de consumo esperado no es uniforme: la consulta de movimientos puede tener mucha más carga que onboarding, mientras transferencias requiere mayor control que throughput puro.

4.4 Desempeño

Se busca baja latencia en lectura y tiempos de respuesta competitivos sin sacrificar consistencia bancaria. La optimización debe ser selectiva: no toda información debe cachearse con el mismo criterio.

4.5 Mantenibilidad y evolución

La arquitectura debe facilitar la incorporación futura de nuevos componentes, canales, productos y servicios externos sin reescritura estructural.

4.6 Auditabilidad y cumplimiento

Debe existir trazabilidad de punta a punta, correlación técnica y de negocio, retención de evidencia y separación entre auditoría, observabilidad y logging aplicativo.

4.7 Operabilidad

La plataforma debe ser observable, medible y operable en producción con tableros, alertas, runbooks, telemetría distribuida y soporte a análisis post-incidente.

5. Supuestos arquitectónicos

Para dar una solución concreta, asumo lo siguiente:

1. El banco es una institución de banca minorista o banca universal con canal digital para personas.
2. El **Authorization Server corporativo ya existe** y puede configurarse, por lo que no se propone desarrollar un motor de autenticación desde cero.
3. El **Core** y el **sistema complementario** pueden estar on-premise, en nube privada o expuestos por conectividad privada híbrida.
4. El onboarding biométrico se resuelve a través de una **plataforma especializada externa**, no como desarrollo propio de algoritmos biométricos.
5. La aplicación móvil es el canal prioritario para onboarding por mejor control sobre cámara, experiencia, dispositivo y biometría local.
6. Las transferencias interbancarias dependen de una red o rail externo que puede responder de forma síncrona o asíncrona según el caso.
7. El canal digital no reemplaza al Core como fuente de verdad de saldos, productos y movimientos.
8. Los objetivos de continuidad propuestos son de referencia arquitectónica; sus valores exactos deben alinearse con la política del banco.
9. La regulación específica dependerá del país; por ello se hablará de **familias regulatorias y estándares de referencia**, no de una ley local única.

6. Principales decisiones de arquitectura

6.1 Arquitectura por capas con servicios de dominio acotados

Decisión. Separar la solución en capa de canal, capa de negocio, capa de integración/anticorrupción, capa de datos y capa operativa.

Justificación teórica. La separación por responsabilidades reduce acoplamiento, favorece cohesión y permite aplicar políticas de seguridad, observabilidad y resiliencia por capa.

Justificación práctica. En banca, el canal evoluciona más rápido que los sistemas legacy. Si ambos quedan mezclados, cada cambio de experiencia impacta integraciones críticas y eleva riesgo operativo.

Alternativas evaluadas.

- Monolito omnicanal: más simple al inicio, pero menos flexible para integración heterogénea.
- Integración directa del front con sistemas backend: descartada por exposición, acoplamiento y debilidad de control.

6.2 BFF por canal

Decisión. Implementar un **Web BFF** y un **Mobile BFF**.

Justificación teórica. El patrón Backend for Frontend reduce el problema del “API genérica inflada” y permite contratos optimizados por canal.

Justificación práctica. Web y móvil tienen necesidades distintas: manejo de sesión, payloads, device context, onboarding, notificaciones push y experiencia en red móvil.

Alternativas evaluadas.

- Un único API omnicanal: más simple, pero con mayor complejidad de contratos.
- Lógica de composición en el frontend: descartada por seguridad, duplicidad y aumento de latencia.

6.3 Core como sistema de registro

Decisión. Mantener el Core como **fuente de verdad** para datos bancarios y transferencias.

Justificación teórica. Los sistemas de registro no deben duplicarse sin una estrategia explícita de consistencia y gobierno de datos.

Justificación práctica. Replicar saldos, movimientos y estados transaccionales en el canal digital introduce riesgo de discrepancias, incidentes regulatorios y soporte costoso.

Alternativas evaluadas.

- Replicar masivamente el Core a una base digital: mejora lectura, pero aumenta complejidad y riesgo de consistencia.
- Canal leyendo solo de cache/read model: inadmisible para operaciones críticas.

6.4 Anti-Corruption Layer hacia sistemas legacy

Decisión. Toda integración con Core, sistema complementario y rail interbancario pasa por adaptadores/ACL.

Justificación teórica. El patrón Anti-Corruption Layer evita que modelos y errores legacy contaminen el dominio del canal digital.

Justificación práctica. Los contratos legacy suelen ser inconsistentes, verbosos o poco expresivos. Encapsularlos facilita versionado, fallback y pruebas.

Alternativas evaluadas.

- Llamadas directas desde servicios de negocio a sistemas externos: más rápidas de construir, pero peligrosamente acopladas.
- ESB central con lógica de negocio: útil como integración corporativa, pero no suficiente por sí solo para proteger el dominio.

6.5 Saga + idempotencia para transferencias

Decisión. Gestionar transferencias con **orquestación de proceso**, control de estados e **idempotency keys**.

Justificación teórica. En flujos distribuidos con múltiples dependencias, 2PC/XA no suele ser la opción más resiliente ni realista.

Justificación práctica. Las transferencias involucran validaciones, posibles confirmaciones diferidas, fallos parciales y reintentos. La idempotencia evita duplicados ante timeout o repetición del usuario.

Alternativas evaluadas.

- Transacción distribuida clásica: descartada por fragilidad y acoplamiento.
- Procesamiento totalmente síncrono sin estado: insuficiente para redes interbancarias y recuperación.

6.6 Outbox para publicación fiable de eventos

Decisión. Publicar eventos a notificaciones, auditoría y cache mediante **Transactional Outbox**.

Justificación teórica. Evita el problema del dual write entre base de datos y bus de eventos.

Justificación práctica. Es crítico que una transferencia confirmada genere de forma fiable auditoría y notificación, incluso si el broker o consumidor tiene una falla temporal.

Alternativas evaluadas.

- Publicar evento dentro de la transacción lógica sin outbox: riesgo de inconsistencia.
- Polling directo desde otros sistemas a la base transaccional: más acoplado y menos limpio.

6.7 Auditoría separada de logs técnicos

Decisión. Diseñar un **Audit Service** y una **Audit DB append-only** separados del logging operativo.

Justificación teórica. Auditoría de negocio y logging técnico responden a objetivos distintos: no deben mezclarse.

Justificación práctica. Un banco debe reconstruir quién hizo qué, cuándo, desde qué canal y con qué resultado, sin depender de logs efímeros de aplicación.

Alternativas evaluadas.

- Usar solo logs centralizados: insuficiente para cumplimiento y trazabilidad formal.
- Hacer que cada microservicio escriba su propia auditoría: posible, pero peor en consistencia y gobierno.

6.8 Cache selectiva para clientes frecuentes

Decisión. Incorporar **Redis/read model** solo para lecturas elegibles.

Justificación teórica. CQRS ligero y cache-aside/read-through son adecuados cuando ciertos datos tienen altísima frecuencia de lectura pero toleran frescura controlada.

Justificación práctica. Perfiles, catálogos, dashboards y últimos movimientos recientes pueden acelerarse; la validación final de transferencias debe seguir consultando sistema fuente.

Alternativas evaluadas.

- No usar cache: simple, pero con peor UX y más carga sobre integraciones.
- Cachear todo: peligroso para saldos y decisiones transaccionales.

6.9 Event-driven para notificaciones y tareas cruzadas

Decisión. Desacoplar notificaciones, auditoría secundaria e invalidación de cache mediante eventos.

Justificación teórica. La mensajería reduce dependencia temporal entre productor y consumidor.

Justificación práctica. El cliente no debe esperar la entrega de un SMS para confirmar una consulta o una transferencia ya aceptada por el sistema.

Alternativas evaluadas.

- Notificación síncrona en línea con la operación: aumenta latencia y fragilidad.
- Batch diferido sin eventos: peor experiencia y menos trazabilidad.

6.10 Nube gestionada con criterio de costo y operación

Decisión. Usar servicios gestionados donde aporten resiliencia y reduzcan carga operativa.

Justificación teórica. En sistemas críticos, conviene externalizar parte de la complejidad no diferenciadora.

Justificación práctica. El banco debe concentrar su complejidad en dominio, seguridad, integración y operación, no en administrar componentes base innecesariamente.

Alternativas evaluadas.

- Infraestructura casi autogestionada: más control, pero mayor costo operativo y riesgo humano.
- Serverless total: útil en algunos componentes, pero no necesariamente óptimo para todos los servicios transaccionales del canal.

7. Justificación de tecnologías, patrones y enfoque

7.1 Modelo de diagramación: C4

El modelo C4 es especialmente adecuado para este ejercicio porque organiza la arquitectura en abstracciones jerárquicas —sistema, contenedores, componentes y código— y, además, es independiente tanto de la notación como de la herramienta. Precisamente eso permite construir una narrativa clara para públicos distintos: negocio, técnicos y equipo de implementación.

7.2 SPA web: Angular como opción preferente

Para la SPA web propongo **Angular** como selección principal.

Por qué lo elijo.

Angular es especialmente conveniente en entornos enterprise con múltiples equipos, fuerte gobierno de front-end, formularios complejos, tipado consistente y necesidad de convenciones estables. En banca, esa estructura ayuda a mantener disciplina arquitectónica, seguridad de UI, testing y evolución controlada.

Alternativa considerada: React.

React es una excelente alternativa, con ecosistema muy amplio y gran disponibilidad de talento. Sin embargo, para un contexto bancario enterprise, Angular suele aportar más uniformidad arquitectónica "out of the box", reduciendo variabilidad entre equipos.

7.3 Aplicación móvil: comparación y selección

Las dos opciones multiplataforma más razonables para este ejercicio son **Flutter** y **React Native**.

Flutter (seleccionada).

La selecciono por cuatro razones: consistencia visual entre plataformas, rendimiento predecible, buen soporte para flujos ricos de onboarding/cámara/animaciones y menor dependencia de bridges JS en jornadas complejas.

React Native (alternativa).

Es una alternativa fuerte cuando existe mucho talento JavaScript/TypeScript y se busca cercanía conceptual con el ecosistema web. Su principal desventaja, para este caso, es que en experiencias intensivas en cámara, biometría y estados complejos puede requerir más cuidado en integración nativa y gobernanza de librerías.

Conclusión.

Para esta solución bancaria escogería **Flutter** como framework móvil principal.

7.4 Backend: coarse-grained services, no microservicios por moda

No recomiendo una explosión de microservicios finos. Propongo **servicios de dominio relativamente gruesos**: Customer & Products, Movements, Transfer Orchestrator, Notification Orchestrator, Onboarding Orchestrator y Audit Service.

Por qué.

En banca, demasiados microservicios elevan complejidad operativa, observabilidad, testing distribuido y riesgo de latencia encadenada. La granularidad elegida equilibra desacoplamiento con gobernabilidad.

7.5 Patrones seleccionados

Los patrones centrales de la propuesta son:

- **BFF**, para contratos por canal.
- **Anti-Corruption Layer**, para encapsular sistemas legacy.
- **Saga/Process Manager**, para transferencias.
- **Transactional Outbox**, para eventos fiables.
- **Cache-aside/read-through** y **read model** para optimización de lectura.
- **Circuit breaker, timeout, retry y bulkhead** para resiliencia.
- **Append-only audit trail** para cumplimiento y trazabilidad.

No son patrones elegidos por catálogo, sino porque cada uno resuelve un riesgo concreto del ejercicio.

8. Diseño de arquitectura propuesto

8.1 Vista general

La arquitectura propuesta puede entenderse como una **plataforma de canales digitales bancarios** que se ubica entre los clientes y los sistemas de registro. Su función no es reemplazar al Core, sino:

- exponer experiencia digital consistente;
- proteger accesos e identidades;
- orquestar procesos de negocio;
- adaptar sistemas heredados;
- registrar evidencia;
- desacoplar notificaciones y procesos cruzados;

- optimizar lectura para mejorar UX.

8.2 Capas lógicas

Capa de acceso y perímetro.

Incluye CDN, protección WAF/DDoS, terminación TLS y API Gateway. Aquí se filtra tráfico, se aplican límites de tasa, validaciones iniciales y versionado de APIs.

Capa de canal.

Contiene la SPA, la app móvil y sus respectivos BFF. Esta capa entiende journeys, composición de respuestas y necesidades propias del dispositivo o navegador.

Capa de negocio.

Aquí viven los servicios que implementan casos de uso: consulta de cliente, movimientos, transferencias, onboarding, notificaciones y auditoría.

Capa de integración.

Aísla el dominio de los contratos del Core, el sistema complementario, el proveedor biométrico, los proveedores de notificación y la red interbancaria.

Capa de datos.

Guarda solo lo que el canal debe poseer: metadatos operativos, estados de procesos, preferencias, outbox, auditoría, evidencias y cache/read models.

Capa operativa transversal.

Seguridad, observabilidad, trazabilidad, alertas, despliegue, recuperación y operación.

8.3 Principios rectores

1. **No exponer sistemas internos al cliente.**
2. **No replicar sin necesidad datos que ya tienen dueño claro en el banco.**
3. **No mezclar auditoría con logging de diagnóstico.**
4. **No hacer síncrono lo que puede desacoplarse sin perder consistencia.**
5. **No usar cache para sustituir validación transaccional.**
6. **No implementar autenticación propia si la organización ya dispone de un producto corporativo.**

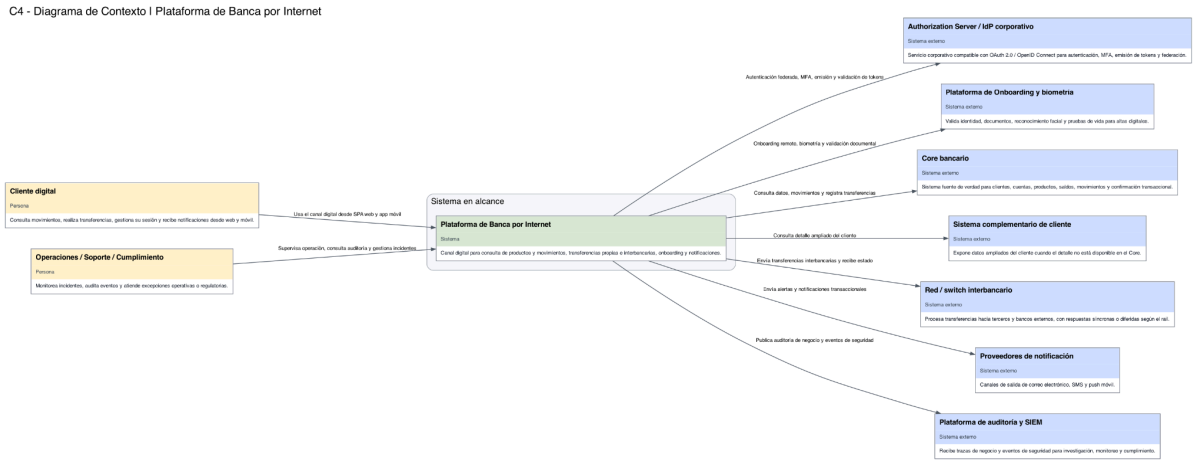
9. Diagrama C4 de Contexto

El diagrama de contexto debe ser entendido por un público mixto, incluso no técnico. Su objetivo es mostrar:

- quién interactúa con el sistema;
- cuáles son los sistemas externos relevantes;
- qué responsabilidades quedan en la plataforma digital;
- qué relaciones críticas existen con identidad, Core, onboarding, notificaciones y auditoría.

En términos C4, este diagrama representa el **software system** dentro de su entorno y deja claro el límite del sistema en alcance. El modelo C4 distingue precisamente el nivel de system context como primer nivel jerárquico de comunicación arquitectónica.

En esta propuesta, el contexto incluye dos actores principales —cliente digital y operaciones/cumplimiento— y siete sistemas externos: IdP corporativo, onboarding biométrico, Core bancario, sistema complementario, red interbancaria, proveedores de notificación y plataforma de auditoría/SIEM.



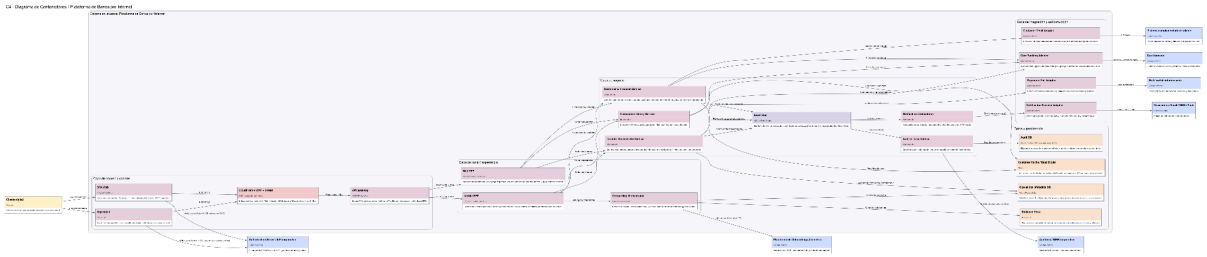
10. Diagrama C4 de Contenedores

El diagrama de contenedores descompone la plataforma en sus unidades de ejecución/deploy o almacenamiento más relevantes:

- canales;
- BFFs;
- servicios de negocio;
- adaptadores;
- bases de datos;
- cache;
- almacenamiento de evidencias;
- bus de eventos.

En términos C4, este nivel permite mostrar la estructura técnica sin caer aún en el detalle de clases o código. El modelo C4 define el container diagram como el segundo nivel jerárquico entre contexto y componentes.

Aquí es donde se hace visible la separación de responsabilidades que más valor aporta al ejercicio: canal, negocio, integración, datos y eventos.

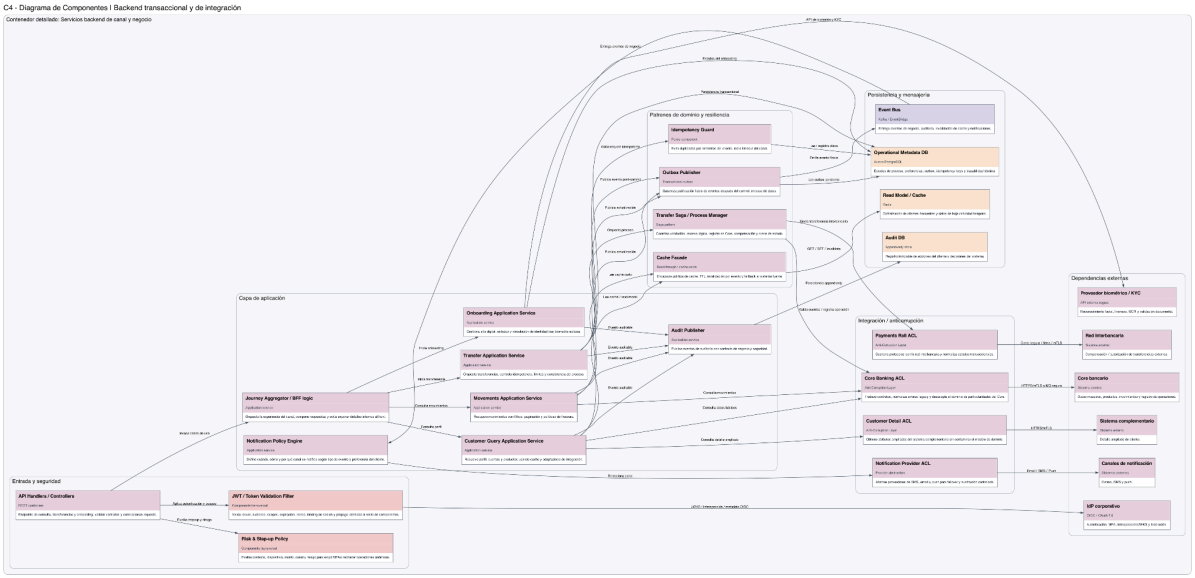


11. Diagrama C4 de Componentes

En C4 estricto, el diagrama de componentes detalla **un contenedor específico**. Para que la propuesta sea útil académicamente, detallo el contenedor backend transaccional y de integración, porque allí se concentran los riesgos más importantes del caso: autenticación contextual, consultas, transfers, onboarding, cache, outbox, Saga, auditoría y ACLs.

Esto es consistente con el modelo C4, que sitúa el component diagram como el tercer nivel jerárquico para explicar cómo se organiza internamente un contenedor concreto.

El diagrama propuesto muestra explícitamente los patrones y controles que suelen diferenciar una arquitectura bancaria madura de una solución “solo funcional”.



Metodología de elaboración de diagramas

Los diagramas de arquitectura se realizaron siguiendo el modelo C4 en sus niveles de contexto, contenedores y componentes. Para su construcción se utilizó Python junto con la librería `graphviz`, lo que permitió definir los nodos, relaciones, límites y agrupaciones de manera programática y reproducible. Esta elección es coherente con el enfoque del modelo C4, ya que C4 es independiente de la herramienta y de la notación, siempre que se respete la semántica de cada nivel.

Proceso de generación

Se implementó un archivo base de utilidades para estandarizar estilos visuales, colores, tipos

de nodos y bordes de sistema. Luego se desarrolló un script independiente para cada diagrama: contexto, contenedores y componentes. Cada script construye un grafo dirigido, define subgrafos para representar límites arquitectónicos y finalmente renderiza el resultado a formato PNG mediante Graphviz. Graphviz soporta agrupación por clusters y salida PNG nativa, lo que permitió producir diagramas limpios y consistentes sin depender de una herramienta manual de dibujo.

Exportación

La exportación se realizó ejecutando los scripts Python, los cuales generan tanto el archivo fuente `.dot` como el archivo final `.png`. De esta forma, los diagramas quedan versionables, trazables y fáciles de modificar si cambia la arquitectura propuesta.

12. Arquitectura de autenticación y autorización

12.1 Recomendación principal

La recomendación principal es:

- **Web SPA: Authorization Code + PKCE**, endurecido mediante **patrón BFF**.
- **Móvil: Authorization Code + PKCE** usando **system browser** o **browser tab**, nunca webview embebida.
- **Identidad:** delegada a un **Authorization Server / IdP corporativo** compatible con OAuth 2.0 y federación.
- **Autorización:** combinación de scopes, claims, políticas de negocio, ownership de recursos y step-up MFA según riesgo.

Esta recomendación se basa en que RFC 8252 establece como mejor práctica que las apps nativas realicen la autorización en un user-agent externo y no en user-agents embebidos; RFC 9700 exige soporte de PKCE, recomienda S256 y prefiere el response type `code` frente a respuestas que exponen tokens en el front channel; además, RFC 6749 prohíbe emitir credenciales de cliente para autenticar clientes user-agent-based o nativos.

12.2 Flujo propuesto para la SPA web

Para una SPA pura, el estándar moderno admite `authorization code + PKCE`. Sin embargo, en un contexto bancario no me quedaría en la lectura mínima del estándar; recomendaría **encapsular el flujo con un BFF**.

Cómo funciona.

1. La SPA redirige al IdP.
2. El usuario autentica y, si corresponde, cumple MFA.
3. El IdP devuelve `authorization code`.
4. El **Web BFF** hace el intercambio del código por tokens.
5. El BFF guarda tokens del lado servidor y la SPA recibe solo una **cookie de sesión HttpOnly, Secure, SameSite**.
6. La SPA consume al BFF, no a los resource servers directamente.

Por qué esta variante es mejor aquí.

- Reduce exposición de tokens en el navegador.
- Minimiza impacto de XSS sobre credenciales de acceso.
- Simplifica revocación, rotación y control central de sesión.
- Facilita token exchange con sistemas internos sin filtrar complejidad al frontend.

Alternativa evaluada.

SPA pública con tokens en browser y `code + PKCE` directo. Es válida técnicamente, pero la considero menos robusta para banca por el riesgo residual de exposición en cliente.

12.3 Flujo propuesto para móvil

La aplicación móvil debe usar **Authorization Code + PKCE** con navegador del sistema o browser tab. RFC 8252 es explícito en privilegiar user-agent externo para apps nativas y en usar PKCE para clientes públicos nativos.

Prácticas recomendadas:

- no usar webviews embebidas;
- almacenar refresh tokens solo en **secure storage** del SO;
- access tokens de vida corta;

- refresh token rotation;
- invalidación ante sospecha de replay;
- atar sesión a device context cuando el producto corporativo lo permita.

RFC 9700 recomienda, para clientes públicos con refresh tokens, usar mecanismos como **sender-constrained tokens** o **refresh token rotation** para detectar replay.

12.4 Autorización

La autorización no debe agotarse en scopes OAuth. Propongo cuatro niveles:

1. **Scopes** de alto nivel: `accounts:read`, `movements:read`, `transfers:write`.
2. **Claims** de identidad y canal.
3. **Políticas de negocio**: límites, horario, estado del cliente, producto habilitado.
4. **Autorización contextual sobre recurso**: verificar que la cuenta consultada o transferida pertenece efectivamente al sujeto o está autorizada.

12.5 Step-up MFA

No todas las acciones requieren la misma fricción. Propongo:

- login con MFA según política del banco;
- step-up para:
  - alta de destinatario;
  - cambio de dispositivo;
  - cambio de datos sensibles;
  - transferencias por encima de umbral;
  - operaciones con señales de riesgo.

NIST SP 800-63B-4 define tres niveles de assurance para autenticadores; para banca digital, es razonable mapear operaciones críticas a un nivel equivalente a **AAL2** o superior según riesgo y regulación local. NIST SP 800-63C-4 complementa este enfoque desde la óptica de federación y assertions.

12.6 Controles adicionales

- exact matching de redirect URIs;
- tokens con audience y lifetime acotados;
- revocación y logout coordinado;
- protección CSRF en el flujo web;
- nonce/state;
- mTLS o red privada para servicios internos;
- jamás usar Resource Owner Password Credentials.

13. Arquitectura de onboarding con reconocimiento facial

13.1 Principio de diseño

El onboarding no debe tratarse como un “anexo” de la autenticación. Es un proceso distinto: **identity proofing + enrollment + bootstrap de identidad digital**. NIST SP 800-63A-4 separa explícitamente el problema de proofing/enrollment del de autenticación posterior, y NIST SP 800-63C-4 cubre la federación/assertions que luego utilizará el ecosistema de identidad.

13.2 Por qué ubicar onboarding en móvil

Selecciono el canal móvil como vehículo principal del onboarding por razones prácticas:

- mejor control de cámara y UX;
- acceso a biometría local del dispositivo;
- mejor captura guiada de documentos y rostro;
- posibilidad de incorporar señales de dispositivo;
- mejor continuidad del journey con push y secure storage.

13.3 Flujo propuesto

1. El usuario instala la app y entra en **modo onboarding**.

2. La app inicia una **sesión anónima controlada** con el Onboarding Orchestrator.
3. La app captura:
   - documento de identidad;
   - selfie o video de vida;
   - datos personales mínimos.
4. El Onboarding Orchestrator invoca al proveedor biométrico/KYC para:
   - OCR;
   - validación documental;
   - face match;
   - liveness.
5. Si aplica, se integran verificaciones adicionales:
   - listas restrictivas;
   - fraude;
   - duplicidad;
   - reglas AML/KYC del banco.
6. Si el resultado es satisfactorio, se crea o precrea el registro de cliente en el banco.
7. Se da de alta la identidad digital/federada en el IdP corporativo.
8. El cliente completa enrolamiento de credenciales y MFA.
9. A partir de ese momento, el usuario ya puede autenticarse por el flujo estándar del canal.

13.4 Justificación

Teórica.

La prueba de identidad debe preceder la autenticación regular: primero se verifica quién es la persona; luego se le emiten o vinculan autenticadores.

Práctica.

Si se mezcla onboarding con login ordinario, se confunden controles, se dificulta la trazabilidad y se complica el manejo de excepciones y revisión manual.

13.5 Tratamiento de biometría

Propongo un manejo conservador:

- almacenar el mínimo indispensable;
- preferir referencias, resultados y evidencias antes que datos biométricos crudos persistidos indefinidamente;
- cifrado reforzado;
- trazabilidad de consentimiento;
- política de retención diferenciada;
- paso a revisión manual ante baja confianza o inconsistencias.

13.6 Alternativas evaluadas

Onboarding web.

Es posible, pero lo considero inferior como primera opción para este caso.

Desarrollo biométrico propio.

No lo recomiendo. El problema no es solo técnico, sino regulatorio, de sesgo, calibración, pruebas y gobierno del modelo. Para un banco, suele ser más razonable integrar un proveedor especializado y gobernarlo adecuadamente.

14. Arquitectura de integración con sistemas Core y sistemas complementarios

14.1 Problema a resolver

El canal digital necesita consultar información de cliente, productos y movimientos desde dos sistemas distintos, y además ejecutar transferencias. El error común sería acoplar directamente cada servicio digital a cada backend legado.

14.2 Solución propuesta

Propongo una **capa de integración y anticorrupción** con tres adaptadores principales:

1. **Core Banking Adapter**
   - consulta datos básicos;
   - consulta movimientos;
   - valida cuentas y registra transferencias.
2. **Customer Detail Adapter**
   - consulta el sistema complementario solo cuando se requiere detalle ampliado.
3. **Payments Rail Adapter**
   - normaliza protocolos y estados con la red interbancaria.

14.3 Justificación

Teórica.

El patrón ACL desacopla semántica del dominio digital respecto de semántica legacy.

Práctica.

Permite:

- mapear errores de negocio;
- aplicar timeouts específicos por sistema;
- introducir circuit breakers;
- testear integraciones en aislamiento;
- cambiar o evolucionar un backend sin reescribir todo el dominio digital.

14.4 Estrategia de consulta

Datos básicos de cliente.

Se consultan preferentemente al Core y, si la vista lo requiere, se complementan con el sistema secundario. Esta composición debe ocurrir en backend, nunca en frontend.

Movimientos.

Se consultan al Core con paginación y filtros. Para UX, puede existir cache corto o read model auxiliar, pero la semántica del origen sigue siendo el sistema bancario.

Transferencias.

El Transfer Orchestrator centraliza:

- validaciones previas;
- límites;
- idempotencia;
- registro de operación;
- interacción con rail interbancario;
- cierre de estado;
- emisión de evento.

14.5 Comportamiento ante degradación

Propongo degradación controlada:

- si falla el sistema complementario, se devuelve respuesta parcial cuando el dato ampliado no sea crítico;
- si falla Core en una consulta, no se “inventa” respuesta con cache antigua salvo que el caso de uso lo admita explícitamente;
- si falla el rail interbancario, la transferencia pasa a estado consistente de error o pendiente, nunca a incertidumbre silenciosa.

14.6 Alternativas evaluadas

- **Front llamando múltiples APIs:** descartado por seguridad, complejidad y duplicidad.

- **Replicación masiva del legado al canal:** útil en algunos escenarios analíticos, pero excesiva para este ejercicio si no se gobierna como programa de datos completo.

15. Arquitectura de notificaciones

15.1 Objetivo

La plataforma debe notificar operaciones y movimientos, con al menos dos canales. Propongo soportar **tres**:

- **push móvil**;
- **correo electrónico**;
- **SMS**.

15.2 Diseño propuesto

Las notificaciones no deben dispararse en línea de manera rígida desde el flujo de negocio. El diseño correcto es:

1. la operación se confirma en el flujo de negocio;
2. el servicio productor publica un evento de negocio;
3. el Notification Orchestrator consume el evento;
4. aplica preferencias, reglas y plantillas;
5. enruta al provider abstraction;
6. el adaptador entrega al proveedor correspondiente;
7. se registran resultado, reintentos y evidencia.

15.3 Justificación

Teórica.

El desacoplamiento temporal reduce fragilidad y mejora elasticidad.

Práctica.

La operación bancaria no debe quedar bloqueada esperando la latencia de un proveedor de SMS.

15.4 Política de uso de canales

Mi recomendación operativa sería:

- **push + email** como combinación por defecto para operaciones normales;
- **SMS** para eventos de criticidad alta, fallback o contextos regulatorios/operativos específicos.

Esto equilibra trazabilidad, experiencia y costo.

15.5 Consideraciones clave

- idempotencia de notificación;
- reintentos exponenciales;
- DLQ;
- plantillas versionadas;
- trazabilidad de entrega;
- separación entre notificación informativa y OTP/segundo factor;
- proveedor abstraction para evitar lock-in.

16. Arquitectura de auditoría y trazabilidad

16.1 Principio rector

En esta solución, **auditoría no es logging**.

- **Logging técnico** sirve para diagnóstico.
- **Observabilidad** sirve para operación.
- **Auditoría** sirve para reconstruir hechos de negocio y seguridad con valor probatorio y de cumplimiento.

16.2 Diseño propuesto

Se propone un **Audit & Trace Service** con dos canales de salida:

1. **Audit DB append-only**, para evidencia estructurada.
2. **SIEM corporativo**, para correlación de seguridad e investigación.

Cada acción relevante debe emitir un evento auditado con, como mínimo:

- timestamp;
- correlation ID / trace ID;
- customer ID;
- user/session/device/channel;
- acción realizada;
- recurso afectado;
- resultado;
- origen IP/red;
- motivo de rechazo si aplica;
- identificador de operación externa.

16.3 Qué se audita

- login/logout;
- fallos de autenticación;
- enrolamiento y step-up MFA;
- onboarding y validación biométrica;
- consultas sensibles;
- altas/bajas de destinatarios;
- transferencias iniciadas, aprobadas, rechazadas, compensadas;
- cambios de preferencias y datos;
- eventos de seguridad.

16.4 Justificación

Teórica.

La auditoría debe ser inmutable, consistente y consultable.

Práctica.

Permite responder a reclamos, disputas, fraude, revisiones internas y regulatorias.

16.5 Alternativas evaluadas

Solo logs centralizados.

Insuficiente.

Cada servicio escribe directo a la Audit DB.

Posible, pero más difícil de gobernar, estandarizar y evolucionar.

16.6 Endurecimiento adicional

Si el banco exigiera no repudio reforzado, podría evolucionarse hacia:

- hash chaining por evento;
- firma de lotes;
- almacenamiento WORM complementario;
- ledger especializado.

No lo pongo como baseline porque aumentaría costo y complejidad; lo propongo como evolución.

17. Estrategia de datos y acceso a datos

17.1 Principio base

El canal digital **no debe convertirse en otro Core**. Por eso distingo tres categorías de datos:

1. **Datos autoritativos bancarios**
   - cuentas, productos, movimientos, saldos, confirmación transaccional;
   - dueño: Core/rail correspondiente.
2. **Datos operativos del canal**
   - preferencias, estado de procesos, device context, idempotency keys, outbox;
   - dueño: plataforma digital.
3. **Datos de soporte**
   - cache, read model, evidencias, auditoría;
   - dueño: plataforma digital con gobierno propio.

17.2 Persistencias propuestas

- **Operational Metadata DB**: Aurora PostgreSQL.
- **Audit DB**: Aurora PostgreSQL o esquema append-only separado.
- **Redis**: cache/read model.
- **S3**: evidencias, comprobantes, artefactos de onboarding y exportaciones.

17.3 Acceso a datos

- lecturas bancarias: vía servicios de negocio + adapters;
- persistencia de canal: vía repositorios internos;
- auditoría: vía servicio dedicado;
- evidencias: vía almacenamiento de objetos;
- cache: vía fachada dedicada, no acceso libre por cualquier componente.

17.4 Consistencia

No todos los casos de uso requieren la misma consistencia.

- **Transferencias**: validación fuerte contra fuente autoritativa.
- **Movimientos recientes para UX**: pueden aceptar cache corto controlado.
- **Perfil ampliado**: puede degradar a respuesta parcial.

Esta diferenciación evita sobrecostear toda la plataforma con consistencia estricta donde no aporta valor.

18. Estrategia de clientes frecuentes / optimización de desempeño / caching

18.1 Problema

El enunciado pide una alternativa de persistencia para clientes frecuentes. La respuesta madura no es “guardar todo en otra base”, sino definir un esquema de optimización selectiva.

18.2 Propuesta

Propongo una combinación de:

- **Redis** como cache read-through / cache-aside;
- **read model ligero** alimentado por eventos para vistas de alta lectura;
- **invalidación por evento** cuando cambie información relevante.

18.3 Qué sí cachear

- perfil resumido;
- catálogo de productos;
- preferencias;
- configuración de canal;
- últimos movimientos de baja criticidad temporal;
- dashboard de inicio.

18.4 Qué no cachear como verdad operativa

- validación final de saldo disponible para transferencias;
- estado final de una transferencia en curso sin verificación;
- datos regulatorios que exijan frescura inmediata;
- decisiones antifraude críticas.

18.5 Justificación

Teórica.

CQRS ligero mejora experiencia sin comprometer integridad si se aplica donde corresponde.

Práctica.

La mayoría de la carga del canal es lectura repetitiva; optimizar esas vistas reduce presión sobre integraciones y mejora percepción de rapidez.

18.6 Alternativas evaluadas

- **Read replica completa del Core:** poderosa, pero más costosa y compleja.
- **Sin cache:** más simple, pero peor desempeño.
- **Cache agresiva global:** riesgosa para decisiones críticas.

19. Alta disponibilidad, tolerancia a fallos, DR y auto-healing

19.1 Objetivo operativo

Propongo como referencia arquitectónica:

- disponibilidad mensual del canal: **99.95% o superior**;
- **RTO objetivo** de referencia: hasta 60 minutos;
- **RPO objetivo** de referencia: hasta 5 minutos para metadatos del canal, y mínima pérdida tolerable para eventos críticos;
- continuidad mejorada para el Core según su propia estrategia corporativa.

19.2 Alta disponibilidad en la región primaria

La plataforma debe desplegarse en **múltiples Availability Zones**. AWS documenta que las AZ son ubicaciones aisladas dentro de una región y recomienda desplegar aplicaciones en múltiples AZ para mantener disponibilidad ante la falla de una de ellas.

Aplicación de ese principio:

- servicios stateless distribuidos en varias AZ;
- balanceo y health checks;
- múltiples instancias/tareas por servicio;
- bases de datos con topología multi-AZ;

- cache con réplicas y failover;
- bus/eventing gestionado con redundancia de zona.

19.3 Base de datos

Aurora documenta almacenamiento replicado en múltiples AZ y opciones de alta disponibilidad regional y cross-region mediante Global Database. Para este diseño, eso la hace adecuada para metadatos operativos y persistencias del canal, no para reemplazar al sistema bancario de registro.

19.4 Tolerancia a fallos lógicos

No basta con Multi-AZ. También propongo:

- **timeouts** por dependencia;
- **reintentos** controlados;
- **circuit breakers**;
- **bulkheads**;
- **colas** para desacople;
- **idempotencia**;
- **fallback parcial** en consultas no críticas.

19.5 DR regional

La estrategia recomendada es **warm standby regional**:

- región primaria activa;
- región secundaria preaprovisionada para componentes críticos;
- replicación de datos operativos y evidencia;
- runbooks y automatización de failover;
- pruebas periódicas de recuperación.

Trade-off.

No recomiendo active-active multi-region desde el inicio, salvo exigencia corporativa muy alta, porque incrementa costo y complejidad de consistencia. Warm standby ofrece mejor balance para el alcance del ejercicio.

19.6 Auto-healing

El auto-healing debe apoyarse en:

- reinicio/reemplazo automático de instancias o tareas no saludables;
- autoscaling horizontal;
- health probes;
- despliegues inmutables;
- rollback automatizado por error budget o fallos de canary.

20. Seguridad, cumplimiento normativo y controles

20.1 Enfoque general

La propuesta adopta **defensa en profundidad**:

- perímetro seguro;
- autenticación fuerte;
- autorización contextual;
- segmentación de red;
- cifrado;
- mínimos privilegios;
- trazabilidad;
- secure SDLC;
- monitoreo de seguridad.

20.2 Controles clave

1. **TLS extremo a extremo** y cifrado en reposo.
2. **Gestión centralizada de secretos y llaves**.
3. **JWT validation robusta** con issuer/audience/scopes.
4. **MFA y step-up** para operaciones de mayor riesgo.
5. **WAF y protección DDoS** en perímetro.
6. **Protección contra automatización abusiva**.
7. **Hardening de APIs**: rate limiting, validación de payload, anti-replay.
8. **Segregación de funciones** y acceso administrativo privilegiado controlado.

9. **Registro de eventos de seguridad** hacia SIEM.
10. **Revisión continua de vulnerabilidades y dependencias**.

20.3 ASVS y secure SDLC

OWASP ASVS proporciona una base formal para definir y verificar controles técnicos de seguridad en aplicaciones web y servicios; además, su versión estable actual es 5.0.0. Lo recomiendo como estándar interno de verificación para front, APIs y seguridad de sesiones.

20.4 PCI DSS y datos de pago

Si la solución entra en contacto con datos de tarjetas o funciones que los involucren, **PCI DSS** debe entrar en el marco de cumplimiento. PCI SSC define PCI DSS como una base de requisitos técnicos y operativos para proteger datos de pago, y mantiene vigente PCI DSS v4.0.1. Si el alcance no incluye datos de tarjetas, conviene diseñar para mantener ese alcance reducido.

20.5 Protección de datos y regulación bancaria

A nivel regulatorio, la arquitectura debe adaptarse a la jurisdicción, pero como mínimo debe contemplar:

- protección de datos personales;
- secreto bancario/confidencialidad;
- KYC/AML y evidencias de onboarding;
- conservación y trazabilidad de operaciones;
- gestión de incidentes;
- continuidad operativa;
- gobierno de terceros.

No afirmo aquí una ley local específica porque depende del país y del regulador supervisor.

20.6 Sobre WAF y fraude

AWS WAF ayuda a limitar tasas altas y proteger disponibilidad, pero su propia documentación advierte que el rate limiting no está pensado como control preciso de tasa. Por ello, en banca no debe confundirse WAF con un verdadero motor antifraude o risk engine transaccional; ambos son complementarios, no equivalentes.

21. Observabilidad, monitoreo, alertamiento y operación

21.1 Principio

Una solución crítica debe ser **observable por diseño**, no “monitoreada después”. OpenTelemetry es especialmente útil aquí porque ofrece un marco vendor-neutral para instrumentar, recolectar y exportar telemetría, y trabaja sobre señales como traces, metrics y logs.

21.2 Estrategia propuesta

Propongo instrumentación end-to-end con:

- **trazas distribuidas**;
- **métricas técnicas y de negocio**;
- **logs estructurados**;
- **correlation IDs / trace IDs**;
- **dashboards por journey**;
- **alertas operativas y de seguridad**;
- **synthetic monitoring** para journeys críticos.

21.3 Qué monitorear

Capa de canal

- disponibilidad de SPA;
- errores JS críticos;
- tiempos de carga;
- latencia de BFF.

Autenticación

- tasa de éxito/fallo;
- MFA challenges;
- step-up conversion;
- replay o anomalías.

Negocio

- p95/p99 de consulta;
- transferencias aceptadas/rechazadas;
- duplicados evitados por idempotencia;
- onboarding completado vs abandonado.

Integración

- latencia por backend;
- timeout rate;
- circuit breaker opens;
- errores por sistema externo.

Mensajería

- lag;
- DLQ size;
- retries;
- throughput.

Datos

- cache hit ratio;
- latency DB;
- replicas lag;
- storage growth.

21.4 SLOs y alertamiento

Recomiendo gobernar la operación con SLOs, por ejemplo:

- disponibilidad del canal;
- latencia de consulta;
- tiempo de aceptación de transferencia;
- tiempo de entrega de notificación;
- tasa de error por journey.

El alertamiento debe distinguir entre:

- incidente operativo;
- degradación parcial;
- anomalía de seguridad;
- problema de proveedor externo.

21.5 Operación real

Además de tableros, hacen falta:

- runbooks;
- on-call claro;
- postmortems;
- game days;
- pruebas de DR;
- capacity review;
- revisión periódica de error budgets.

22. Estrategia de nube (AWS)

22.1 Selección de nube

Elijo **AWS** para esta propuesta.

22.2 Justificación

AWS Well-Architected estructura el diseño alrededor de seis pilares: excelencia operativa, seguridad, confiabilidad, eficiencia de desempeño, optimización de costos y sostenibilidad. Esa matriz es particularmente útil para defender una arquitectura bancaria porque obliga a balancear seguridad, resiliencia y costo, no solo funcionalidad.

Además, AWS documenta una infraestructura regional con Availability Zones aisladas y servicios gestionados que encajan muy bien con esta solución de canales: perímetro, API management, cómputo, base de datos, cache, storage y telemetría.

22.3 Mapeo lógico propuesto

Perímetro y acceso

- CloudFront
- WAF
- Shield
- API Gateway

Cómputo

- BFFs y servicios de negocio sobre **ECS/Fargate** como opción inicial preferente por menor carga operativa.
- Si el banco ya tiene una plataforma Kubernetes madura, el diseño es trasladable a EKS.

Datos

- Aurora PostgreSQL

- ElastiCache Redis
- S3 para evidencias

Mensajería

- Event bus lógico implementable con EventBridge y/o mensajería desacoplada;
- si el banco ya opera Kafka corporativo, el diseño admite MSK.

Observabilidad

- telemetría con OpenTelemetry hacia stack corporativo / backend observability elegido.

22.4 Por qué ECS/Fargate como preferencia inicial

Para este alcance, favorezco **ECS/Fargate** porque:

- reduce carga de administración de cluster;
- es suficiente para servicios stateless del canal;
- simplifica escalado y operación;
- reduce complejidad de plataforma frente a Kubernetes si no hay un equipo ya maduro que lo justifique.

22.5 Conectividad híbrida

Dado que el Core puede no estar en AWS, la arquitectura debe asumir **conectividad privada híbrida** y no exponer integraciones legacy a internet pública.

23. Manejo de costos y trade-offs

23.1 Principio

En banca, “lo más seguro” y “lo más caro” no siempre son sinónimos, pero “lo más barato” casi nunca es una decisión suficiente. La estrategia correcta es **gastar donde reduce riesgo material** y contener costo donde el valor marginal es bajo.

23.2 Principales generadores de costo

- mensajería SMS;
- almacenamiento y retención de observabilidad;
- DR multi-región;
- servicios gestionados de alta disponibilidad;
- proveedor de biometría/KYC;
- WAF/DDoS avanzado;
- tráfico entre zonas/regiones;
- sobrefragmentación de microservicios.

23.3 Decisiones de optimización

1. **No duplicar el Core innecesariamente.**
2. **Usar cache selectiva**, no replicación masiva.
3. **Preferir push/email** sobre SMS cuando el negocio lo permita.
4. **Warm standby** en DR en lugar de active-active desde el día uno.
5. **Servicios gestionados** donde reduzcan carga operativa y riesgo humano.
6. **Observabilidad por capas de retención**, no retener todo al máximo detalle por el mismo tiempo.
7. **Granularidad moderada de microservicios**, evitando complejidad innecesaria.

23.4 Trade-offs relevantes

BFF adicional vs simplicidad.

Más componentes, pero mejor seguridad y experiencia por canal.

Aurora/servicios gestionados vs infraestructura autogestionada.
Mayor costo unitario, menor costo operativo/riesgo.

Warm standby vs active-active.
Menor costo y complejidad, a cambio de mayor RTO que active-active.

Event bus desacoplado vs flujo síncrono simple.
Mayor diseño inicial, mejor resiliencia y escalabilidad.

24. Riesgos de arquitectura y mitigaciones

1. **Latencia o indisponibilidad del Core**

Mitigación: timeouts, circuit breakers, cache selectiva, degradación parcial y observabilidad por dependencia.

2. **Duplicidad de transferencias por reintentos**

Mitigación: idempotency keys, control de estado transaccional, Sagas y auditoría.

3. **Exposición de tokens o sesión en canal web**

Mitigación: BFF, cookies HttpOnly/SameSite, PKCE, CSP, endurecimiento contra XSS/CSRF.

4. **Falsos rechazos o errores en onboarding biométrico**

Mitigación: liveness, thresholds calibrados, revisión manual, evidencia y fallback operativo.

5. **Crecimiento excesivo de complejidad por microservicios**

Mitigación: servicios coarse-grained y gobierno de dominios.

6. **Deriva entre auditoría y logs operativos**

Mitigación: servicio y modelo de auditoría dedicados.

7. **Vendor lock-in**

Mitigación: abstraer servicios lógicos, usar ACLs, OTel y patrones portables.

8. **Costos crecientes en notificaciones y observabilidad**

Mitigación: políticas de canal, retención escalonada, dashboards con cardinalidad controlada.

9. **Riesgo regulatorio por almacenamiento de biometría y evidencias**

Mitigación: minimización de datos, cifrado, retención definida y consentimiento explícito.

25. Conclusión final

La propuesta presentada resuelve el ejercicio con una arquitectura **enterprise, realista y defendible** para banca digital. La clave no está en "usar muchas tecnologías", sino en organizar correctamente responsabilidades: canales desacoplados, autenticación federada, onboarding biométrico gobernado, Core como fuente de verdad, integración protegida por ACL, transferencias con control de proceso e idempotencia, notificaciones event-driven, auditoría formal, cache selectiva, observabilidad real y resiliencia cloud.

Desde un punto de vista de comité de arquitectura, considero que esta propuesta es sólida porque:

- satisface explícitamente los requerimientos del problema;
- justifica sus decisiones con teoría y práctica;
- muestra trade-offs reales;
- distingue correctamente seguridad, auditoría, integración y operación;
- evita errores frecuentes como exponer tokens en el navegador sin mitigación, duplicar el Core o confundir logs con auditoría;
- aterriza la solución en una nube concreta sin perder portabilidad conceptual.

Mi recomendación final sería presentar esta arquitectura como una **versión objetivo inicial (target architecture v1)**, dejando preparadas evoluciones futuras como passkeys, antifraude avanzado, ledger reforzado para auditoría o active-active regional si el apetito de riesgo del banco lo exigiera.

26. Referencias

1. C4 model. *The C4 model for visualising software architecture*. C4 model.
2. Hardt, D. (Ed.). *RFC 6749: The OAuth 2.0 Authorization Framework*. Internet Engineering Task Force (IETF).
3. Sakimura, N., Bradley, J., Jones, M., de Medeiros, B., & Mortimore, C. *RFC 7636: Proof Key for Code Exchange by OAuth Public Clients*. Internet Engineering Task Force (IETF).
4. Denniss, W., & Bradley, J. *RFC 8252: OAuth 2.0 for Native Apps*. Internet Engineering Task Force (IETF).
5. Bradley, J., Labunets, A., & Fett, D. *RFC 9700: Best Current Practice for OAuth 2.0 Security*. Internet Engineering Task Force (IETF).
6. National Institute of Standards and Technology (NIST). *SP 800-63A-4: Digital Identity Guidelines – Identity Proofing and Enrollment*.
7. National Institute of Standards and Technology (NIST). *SP 800-63B-4: Digital Identity Guidelines – Authentication and Authenticator Management*.
8. National Institute of Standards and Technology (NIST). *SP 800-63C-4: Digital Identity Guidelines – Federation and Assertions*.
9. Amazon Web Services. *AWS Well-Architected Framework*.
10. Amazon Web Services. *AWS Regions and Availability Zones*.
11. Amazon Web Services. *AWS Availability Zones*.
12. Amazon Web Services. *High availability for Amazon Aurora*.
13. Amazon Web Services. *Using Amazon Aurora Global Database*.
14. Amazon Web Services. *AWS WAF Documentation*.
15. Amazon Web Services. *Rate-based rule caveats in AWS WAF*.
16. OpenTelemetry. *OpenTelemetry Documentation*.
17. OWASP Foundation. *OWASP Application Security Verification Standard (ASVS)*.
18. PCI Security Standards Council. *PCI Data Security Standard (PCI DSS)*.
19. PCI Security Standards Council. *Standards Updates: PCI DSS v4.0.1*.
20. graphviz. *User Guide — graphviz documentation*.
21. graphviz. *Graphviz documentation*.